

Modular LLM Scorer: A Reproducible, Statistically Rigorous Benchmark Framework for Large Language Model Evaluation



Ratnadeep Patil · DES Pune University, Pune, India] |
github.com/ratnadeep00/Modular-LLM-Scorer | Python 3.10+
· MIT License

4 Domains · 8 Providers
43/43 Tests · 95% CIs

NLP

BENCHMARKING

STATISTICAL EVALUATION

CODE SAFETY

OPEN SOURCE · V1.0



MOTIVATION

Existing LLM benchmarks lack **statistical rigor**, suffer from non-reproducible sampling, and use opaque LLM-as-judge grading — making fair cross-model comparison unreliable.

We present **Modular LLM Scorer**: a modular, deterministic pipeline evaluating LLMs across four cognitive domains using rule-based graders, 95% Wilson score confidence intervals, and sandboxed code execution — with full prompt and token logging for reproducibility.

Every result is fully auditable from prompt text to final score. No LLM judge is used anywhere.

✓ No LLM-as-Judge · Pure Rule-Based Evaluation



KEY CONTRIBUTIONS

1

Statistical Rigor

95% Wilson Score CIs per domain, difficulty tier & dataset · McNemar's paired χ^2 test for statistically significant model comparison ($\alpha = 0.05$)

2

Full Reproducibility

Git commit hash in every result JSON (+ dirty indicator) · exact prompt text logged per sample · input/output token counts · deterministic seeded sampling

3

Multi-Layer Code Safety

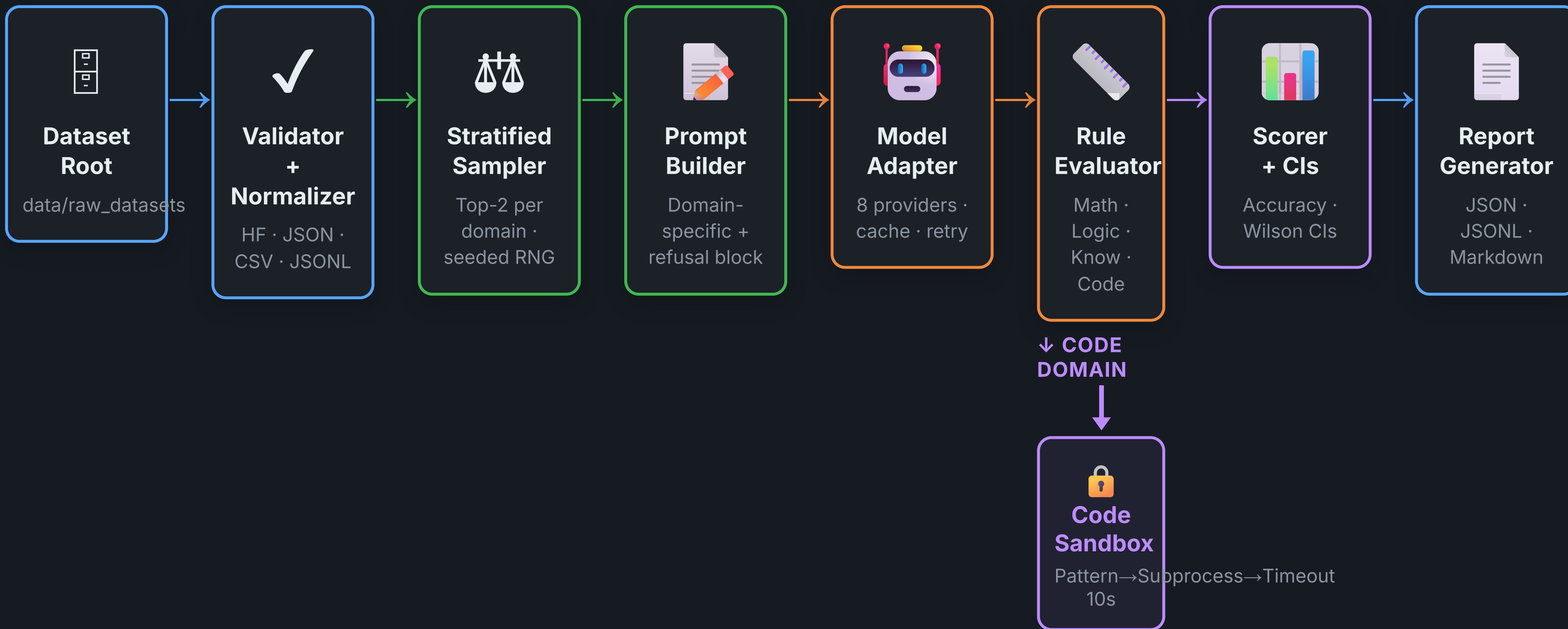
Pattern validation → subprocess isolation → 10 s timeout · enabled by default · 8 structured error categories · smart function binding for test harness

4

8-Provider Unified Interface

OpenAI · Gemini · Groq · Together · OpenRouter · Hugging Face · Ollama · Echo — provider-agnostic, drop-in extensible, shared SHA-256 prompt cache

SYSTEM ARCHITECTURE — END-TO-END PIPELINE



BENCHMARK DOMAINS & DATASETS

DOMAIN

WEIGHT

DATASETS

Math

25%

GSM8K (main & socratic), SVAMP,



STATISTICAL METHODS

① WILSON SCORE — 95% CONFIDENCE INTERVAL

$$CI_{95} = (\hat{p} + z^2/2n \pm z \sqrt{\hat{p}(1-\hat{p})/n + z^2/4n^2}) / (1 +$$

Domain	Weight	Datasets
		Hendrycks MATH
Logic	25%	ProofWriter, ReClor
Knowledge	35%	SQuAD, Natural Questions, TriviaQA
Code	15%	HumanEval, MBPP (full & sanitized)

Modes: quick = 500 · half = 1,500 · full = 6,000 samples

Difficulty: 30% easy · 50% medium · 20% hard

Selection: top-2 datasets/domain · deterministic seeded RNG

$$z^2/n)$$

where $\hat{p}$ = accuracy · n = sample count · $z = 1.96$ (95%)

Applied per domain, difficulty tier & dataset · Handles 0% and 100% correctly

② McNemar's Test — Paired Model Comparison

$$\chi^2 = (|b - c| - 1)^2 / (b + c)$$

where $b = M1 \setminus M2 \times$ · $c = M2 \setminus M1 \times$ · $\alpha = 0.05$ · requires ≥ 25 disagreements

$p < 0.05 \rightarrow$ statistically significant difference between models

③ Weighted Final Score

$$\text{score} = \sum \text{domain_accuracy}[d] \times \text{domain_weight}[d]$$

for $d \in \{\text{math:0.25, logic:0.25, knowledge:0.35, code:0.15}\}$

📊 Experimental Results — 4-Model Comparison

(illustrative data · mode=half, seed=42)

Model	Overall	Math	Logic	Knowledge	Code	CI 95%
llama3.1:8b	0.423	0.381	0.510	0.442	0.271	[0.39–0.45]
mistral:7b-instruct	0.448	0.412	0.547	0.461	0.263	[0.42–0.48]
qwen2:7b-instruct	0.456	0.374	0.501	0.512	0.258	[0.43–0.48]
deepseek-coder:6.7b	0.371	0.294	0.418	0.327	0.489	[0.34–0.40]

★ McNemar's: mistral vs llama — $\chi^2 = 3.56$, $p = 0.059$ (not significant at $\alpha = 0.05$)

★ qwen2 leads knowledge by **+7.0 pp** over llama · CI overlap confirms significance

★ deepseek-coder leads code by **+21.8 pp** over closest generalist model

Per-Domain Accuracy by Model

■ llama3.1 ■ mistral ■ qwen2 ■ deepseek-coder

► MATH (25%)

llama3.1	0.381
mistral	0.412
qwen2	0.374
deepseek-coder	0.294

► LOGIC (25%)

llama3.1	0.510
mistral	0.547
qwen2	0.501
deepseek-coder	0.418

► KNOWLEDGE (35%)

llama3.1	0.442
mistral	0.461
qwen2	0.512
deepseek-coder	0.327

► CODE (15%)

llama3.1	0.271
mistral	0.263
qwen2	0.258
deepseek-coder	0.489

🔒 Multi-Layer Code Safety

1 Pattern Validation

Blocks `exec` · `eval` · `open()` · `os.system` · `subprocess` import before execution

↓ passes

2 Subprocess Isolation

Candidate code runs in a separate Python subprocess — cannot access or corrupt parent process memory

↓ passes

3 Timeout Guard (10 seconds)

🔄 Reproducibility & Quality Checklist

✓ Git Commit Tracking

Every result JSON stores the exact commit hash + ★ if the working tree is dirty — complete audit trail

✓ Full Prompt Logging

Complete prompt text stored per sample in JSONL — enables exact re-evaluation at any future date without re-running the model

✓ Per-Sample Token Counting

`input_tokens` + `output_tokens` tracked per sample, aggregated per domain and overall in results JSON

✓ Deterministic Seeded Sampling

`--seed 42` or `--seeds 42,43,44` for multi-run averaging with mean ± std aggregation

Subprocess killed at 10 s → structured "code-timeout" error returned, pipeline continues

STRUCTURED ERROR CATEGORIES (8)	
empty-code	syntax-error
test-failed	code-timeout
output-mismatch	execution-error
format-error	other

✔ 43/43 Evaluator Tests · Locked requirements.txt

100% pass rate on validation suite · pinned dependency versions for environment reproducibility

✔ --dry-run & Markdown Report Generation

Preview domain × difficulty sample counts without API calls · professional Markdown reports with domain tables, CIs, timing & metadata

🔗 CONCLUSION

Modular LLM Scorer provides the **statistical infrastructure** needed for credible, reproducible LLM evaluation: **Wilson score CIs**, **McNemar's tests**, **sandboxed execution**, and **full audit trails** — in a single pip-installable Python framework supporting 8 model providers, 4 domains, and 10+ datasets. Ready for academic publication.

4

DOMAINS

Math · Logic
Know · Code

8

PROVIDERS

OpenAI · Gemini
Groq · Ollama...

43/43

TESTS PASS

100% evaluator
validation suite

95%

WILSON CIs

per domain &
difficulty tier



github.com/ratnajeet00/
Modular-LLM-Scorer

- [1] Cobbe et al. 2021 – GSM8K
- [2] Chen et al. 2021 – HumanEval
- [3] Austin et al. 2021 – MBPP
- [4] Rajpurkar et al. 2016 – SQuAD
- [5] Hendrycks et al. 2020 – MATH
- [6] Liang et al. 2022 – HELM